Clase 7

@September 17, 2026

Introducción

En esta clase creamos el nuevo shader llamado Cube.hlsl. Y además modificamos todo Engine.cpp para poder hacer un cubo de colores que gira. Fue una clase corta pero muy densa. El video de 20 minutos me lo vi en una hora.

Desarrollo

Nuevas Librerías

En `Engine.cpp` agregamos dos librerías para dar el salto del 2D al 3D:

`<DirectXMath.h>`

Es la librería oficial de matemáticas de DirectX optimizada para hardware moderno (utiliza instrucciones SIMD de la CPU). Proporciona estructuras de vectores (`XMVECTOR`), matrices (4×4 mediante `XMMATRIX`) y funciones de transformación espacial (`XMMatrixRotationX`, `XMMatrixLookAtLH`, `XMMatrixPerspectiveFovLH`).

`<chrono>`

Es parte de la librería estándar de C++ (`std::chrono`) para medición de tiempo de alta precisión. Permite calcular el tiempo transcurrido (*delta time* / `elapsedSeconds`) entre fotogramas para que la rotación y movimiento del cubo sean fluidos e independientes de los FPS del monitor.

Conceptos

¿Por qué se usa `struct alignas(16) TransformBuffer`?

Las tarjetas gráficas (GPU) procesan datos en bloques de 16 bytes (128 bits, correspondiente a vectores de 4 flotantes `float4`). DirectX 11 exige estrictamente que todos los *Constant Buffers* enviados a la GPU estén

alineados a múltiplos de 16 bytes en la memoria RAM/VRAM. Usar `alignas(16)` fuerza al compilador de C++ a empaquetar la estructura en un bloque alineado a 16 bytes, evitando fallos de lectura o cierres inesperados en la GPU.

Rasterizador

En esta parte transformamos el 3D en pixeles discretos.

Entre sus funciones principales están:

1. Determinar qué píxeles cubre cada triángulo.
2. Realizar el **Culling** (descartar caras traseras que no miran hacia la cámara).
3. Aplicar el **Fill Mode** (dibujar el objeto sólido o en malla de alambre / *wireframe*).

¿Por qué cambiamos a Renderizado Indexado (`DrawIndexed`)?

Un cubo tiene **8 vértices único en el espacio**, pero está formado por **12 triángulos** (2 por cada una de sus 6 caras).

Sin índices (`Draw`): Habría que enviar 12 × 3 = 36 vértices a la GPU, duplicando datos repetidos innecesariamente.

Con índices (`DrawIndexed`): Se definen solo los 8 vértices en el `VertexBuffer` y se crea un `IndexBuffer` con 36 enteros (`uint16_t`) que le indican a la GPU qué vértices conectar. Esto reduce de manera drástica el ancho de banda y el consumo de VRAM.

Depth Stencil Buffer

Hicimos el Buffer de Profundidad y su respectiva **Vista**, esenciales para determinar qué objetos están delante de otros en 3D.

```
D3D11_TEXTURE2D_DESC depthBufferDescription{};

depthBufferDescription.Width = engine.width;
depthBufferDescription.Height = engine.height;
depthBufferDescription.MipLevels = 1;
depthBufferDescription.ArraySize = 1;
depthBufferDescription.Format = DXGI_FORMAT_D24_UNORM_S8_UI
```

```
NT;

depthBufferDescription.SampleDesc.Count = 1;
depthBufferDescription.SampleDesc.Quality = 0;
depthBufferDescription.Usage = D3D11_USAGE_DEFAULT;

depthBufferDescription.BindFlags = D3D11_BIND_DEPTH_STENCI
L;
```

Explicación Línea por Línea:

Configuración de la Textura 2D de Profundidad:

- `Width` / `Height`: Define que la textura de profundidad tenga el mismo tamaño en píxeles que la ventana del motor.
- `Format = DXGI_FORMAT_D24_UNORM_S8_UINT`: Indica que cada píxel de esta textura guardará 24 bits para la profundidad (Z-buffer, valores entre 0.0 y 1.0) y 8 bits para operaciones de *stencil* (máscaras/plantillas).
- `BindFlags = D3D11_BIND_DEPTH_STENCIL`: Le notifica a DirectX que esta textura no se usará como una imagen común, sino como un buffer de prueba de profundidad.

Creación del Recurso en la GPU:

```
result = engine.device->CreateTexture2D(&depthBufferDescrip
tion, nullptr, &engine.depthStencilBuffer);
```

Solicita a la tarjeta gráfica reservar la memoria VRAM física necesaria para la textura de profundidad con las especificaciones configuradas.

Creación de la Vista:

```
result = engine.device->CreateDepthStencilView(engine.depth
StencilBuffer, nullptr, &engine.depthStencilView);
```

En DirectX 11, las canalizaciones gráficas no leen texturas directamente; necesitan una **Vista** (*View*). `CreateDepthStencilView` crea el punto de enlace que permite al rasterizador escribir y comparar valores Z píxel por píxel durante el renderizado.

Transformación de la Matriz MVP

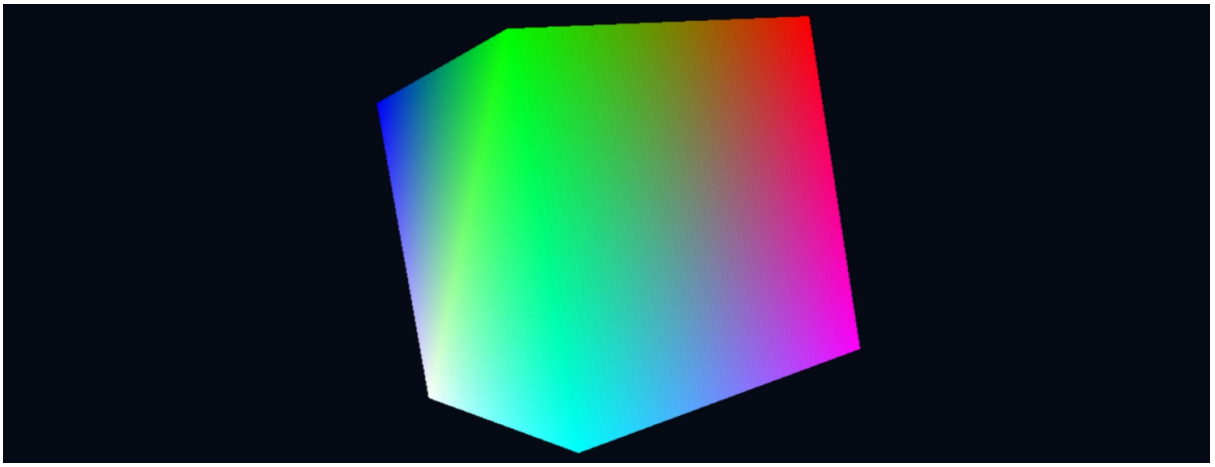
El shader `Cube.hlsl` y el bucle de renderizado en `Engine.cpp` ahora se coordinan mediante la matriz **MVP** (*Model-View-Projection*):

Transform = World x View x Projection

```
  [ Vértice 3D Local ]
           |
           v
  [ Matriz de Mundo (World) ] ----> Escala, Rotación (X, Y) y Posición en el espacio
           |
           v
  [ Matriz de Vista (View) ] -----> Posición y orientación de la Cámara (LookAt)
           |
           v
  [ Matriz de Proyección ] -------> Perspectiva FOV, Aspect Ratio y rango Z (0.1 a 10
0.0)
           |
           v
  [ Shader VSMain (HLSL) ] -------> mul(position, worldViewProjection)
```

En `Render()`, la función `UpdateSubresource` toma esta matriz combinada y la transmite al `transformBuffer` en la GPU antes de ejecutar `DrawIndexed(36, 0, 0)`.

CUBO DE COLORES QUE GIRA



El paso de 2D a 3D implica una curva de aprendizaje pronunciada debido a la cantidad de conceptos abstractos que deben configurarse antes de ver un solo píxel en pantalla (alineación de memoria, buffers de profundidad, vistas, matrices de proyección e índices).

Conclusión

La verdad si me sentí super estresado en esta clase, me fui a hacer el equivalente de berrinche en un adulto funcional y regresé preparado mentalmente para el proceso. Me di cuenta de que ver el código y copiarlo me llena de enojo ya que no le encuentro sentido. Pero ir lento y acompañarme de la explicación del profe le otorga las conexiones entre los conceptos que me faltaban y me hace disfrutar la clase un poco más porque hace sentido. Esto significa que el sentido literalmente es una luz que nos va a salvar de la perdición y la muerte.

Glosario

Rasterizador

El **Rasterizador** (*Rasterizer*) es la etapa del pipeline gráfico encargado de convertir la geometría vectorial tridimensional (triángulos definidos en coordenadas espaciales) en píxeles discretos dentro de una pantalla 2D.

Entre sus funciones principales están:

1. Determinar qué píxeles cubre cada triángulo.
2. Realizar el **Culling** (descartar caras traseras que no miran hacia la cámara).
3. Aplicar el **Fill Mode** (dibujar el objeto sólido o en malla de alambre / *wireframe*).

Benchmarking

El **Benchmarking** en gráficos computacionales es el proceso de medir y evaluar el rendimiento del motor o la GPU en tiempo de ejecución. Consiste en registrar métricas como la tasa de fotogramas por segundo (FPS), el tiempo de renderizado por fotograma en milisegundos ms, y el uso de memoria para

identificar cuellos de botella (*bottlenecks*) en el procesamiento de vértices o píxeles.